Week 11 Level 2: Intermediate

Project Studio 2

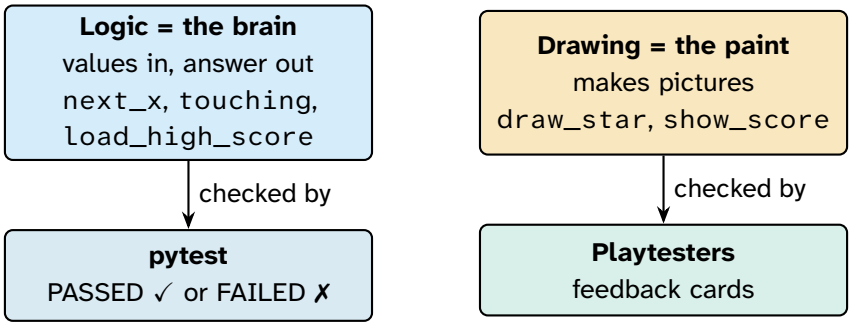
Tests check the brain of your program, playtesters check the fun, and polish makes it work for everyone.

What you will learn

- How to add data to your project: a list, a dictionary and a file that saves and loads (Milestone M3).
- How to write at least three pytest tests for your own functions.
- How to run a playtest and turn feedback into a bug list.
- How to check your project for accessibility, then polish it (Milestone M4).

The big idea: test the brain, playtest the fun

Your program has two kinds of functions. **Logic** functions (the brain) take values in and give an answer back with `return`: pytest can check those automatically. **Drawing** functions (the paint) make pictures: a computer cannot tell whether your star looks nice, but a playtester can.



Two tricks make your functions easy to test:

1. Put the start-up code (make the screen, start the loop) under `if __name__ == "__main__":`. Then pytest can borrow your functions without starting the game.
2. Give file functions the file name as a parameter. Then the game and the tests can use different files, for example `highscore.txt` for playing and `test_scores.txt` for testing, and your tests never wipe your real high score.

Test a **normal case** (an everyday value) and **edge cases** (values at the very limit: the edge of the screen, a score of `0`, a file that does not exist yet). Bugs love to hide at the edges.

Exercise 11.1: Add your data (M3)

Your project needs all three. Write where each one lives in your code.

Data	Its name in my code	What it remembers
a list		
a dictionary		
a file		

☐ I closed the program, ran it again, and the saved data came back.

☐ If the file does not exist yet, my program does not crash.

Exercise 11.2: Write three tests

Plan your tests first. Pick functions that `return` something.

Function	Input	Expected answer	Normal or edge?
`next_x`	`280, 1`	`280` (stays on screen)	edge

Make a new file in the same folder as your game. Its name must start with `test_`, for example `test_mygame.py`. Each test is a function whose name starts with `test_`:

test_mygame.py (part)

```
from mygame import load_high_score, next_x, save_high_score

def test_stops_at_the_right_edge():
    assert next_x(280, 1) == 280

def test_high_score_round_trip():
    save_high_score(42, "test_scores.txt")
    assert load_high_score("test_scores.txt") == 42
```

To run them: press F5 on your game once, close its window, then in Thonny choose **Tools > Open**

system shell... and type (use `python3` if `python` is not found):

>_ Terminal

```
python -m pytest test_mygame.py -v
```

PASSED means the function gave the expected answer. **FAILED** means it did not: read the line starting with E to see what came out instead. A test that fails is not a disaster, it is a bug you found before a player did!

☐ I have at least 3 tests and they all say PASSED.

☐ I broke my code on purpose once, and a test turned to FAILED. (Then I fixed it back.)

Exercise 11.3: Playtest with feedback cards

Swap computers with another team (or ask a family member) and play their project for 4 minutes. Fill in a card for them. Be kind and specific.

Playtest feedback card

I liked...

I was confused when...

A bug I saw (what I did / what happened / what I expected):

One idea...

Back at your own project, turn the cards into a **bug list**. Mark each line MUST (it crashes or you cannot play), SHOULD (it is confusing) or COULD (a nice idea). Fix the MUST lines first.

What I did	What happened	What I expected	MUST / SHOULD / COULD

Exercise 11.4: Accessibility review and polish (M4)

Make your project work for as many people as possible. Check off each line or fix it.

- ☐ My colors come from the Okabe–Ito palette (`#E69F00`, `#56B4E9`, `#009E73`, `#0072B2`, `#D55E00`, `#CC79A7`).
- ☐ Nothing depends on color alone: good and bad things also have different shapes, sizes or labels.
- ☐ Text is big enough to read from a chair (font size 18 or more).
- ☐ The keys are explained on the screen, for example "← → to move".
- ☐ Nothing flashes fast (more than 3 times a second can hurt some people's eyes).
- ☐ There is an easy start: a slow first level or an easy setting.

One change I made:

Vocabulary box

playtest letting someone else use your program while you watch and take notes.

normal case a test with an everyday value, like moving from the middle of the screen.

edge case a test with a value at the very limit, like the edge of the screen, zero, or a missing file.

accessibility designing so that everyone can use it, including people who see colors differently or read more slowly.

polish small finishing touches that make a working program clearer and nicer to use.

bug list a list of problems found in testing, sorted so the most important ones get fixed first.

Reflection

Which test are you proudest of? What bug did it catch, or could it catch?

What did a playtester notice that you had not noticed?

Week 11 Level 2: Intermediate

Project Studio 2

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Three passing tests and one fix (about 10 minutes)

1. Open your project folder in Thonny. Make sure you have a file called `test_mygame.py` (or `test_` plus your game's name) with **at least 3 tests**.
2. Run your game once with F5, close its window, then **Tools > Open system shell...** and type `python -m pytest -v` (or `python3 -m pytest -v`).
3. Copy the last line of the output here (for example "3 passed in 0.05s"):

   _____

4. Pick the top MUST line from your playtest bug list and fix it. What did you fix?

   _____

★★ Challenge (want more?): Test first

Find a bug or a missing feature in a logic function. Write the test **first** and run it: it should say FAILED. Then change your code until it says PASSED. Write your test here:

Why is it useful to see the test fail first?

_____

Creative (make it yours): How to play

Add a "how to play" message to your project's screen (with `write` and a font size of 18 or more), or write it on a card to place next to the computer at the showcase. Include: the goal, the keys, and how the game ends (or, for an Art Studio, how to draw, save and load). Test it on a family member: can they start without asking you anything?

Self-check: tick each box when it is true

- ☐ At least 3 of my tests say PASSED.
- ☐ My project saves something to a file and loads it again.
- ☐ I fixed at least one bug from the playtest.
- ☐ I checked the accessibility checklist in workbook chapter 11 (colors, shapes, text size, keys explained, no flashing).
- ☐ High-score names and saved files use nicknames only.